- First, read up on the Linux Completely Fair Scheduler (CFS) algorithm to get a sense of what it does.
- For now, ensure that your tests on a single core using the taskset command (e.g. taskset -c <core-id> ./sched_test ...). Starting with num_hogs = 0, and increasing it to 4, 8, 16, and so on, does the interactive process always remain relatively stable? At what point do you start seeing instability (longer jitters, more frequent jitters?).
    - **I started to see instability after adding around 100 cpu hogs.**
- Human perception: Presumably you type faster than 300ms per keypress. Change the CLICK_INTERVAL_MS constant in interactive.c to more closely match your typing speed.
    - At what jitter value (in ms) do you personally start to notice our server feeling "laggy"?
        - **3.5 milliseconds.**
    - Note how many CPU hogs did you have to deploy on a single core for you to observe this lag.
        - **30 hogs**
    - What if you ran your program without the taskset -c <core-id> prefix so that the hogs are scheduled on multiple cores? Does that help with lag? At what point (i.e., how many hogs did it take) does the lag return, if ever, in your testing?
        - **It definitely helps. It doesn't return to the same jitter value until there are 250 cpu hogs.**
- Based on both your experimental results and the CFS reading:
    - What does the Linux CFS appear to prioritize (e.g., **fairness**, throughput, latency, or a combination)?
        - **It strongly prioritises fairness. (It is named the Completely Fair Scheduler, after all). That being said, it could be argued as a combination as it does try to take latency and throughput into account by how it prioritises processes.**
    - How does the scheduler maintain responsiveness for the interactive process, even in the presence of CPU-bound hogs? Describe the mechanism in terms of how CPU time is distributed among processes under CFS.
        - **The CFS uses something called the red-black tree. It's time-ordered, self-balancing, and tree operations are O(log n).**
        - **Tasks that have the lowest virtual runtime are stored on the left side of the tree, while those with higher virtual runtimes are on the right. Tasks with the lowest virtual runtime are scheduled next (This is for fairness).**
        - **After using the CPU, the process adds its execution time to its virtual runtime and is then given back to the tree (if it can still be run). This allows tasks on the left to be run first, but prevents starvation by causing contents of the right side of the tree to eventually move to the left.**